SIMVA: cambios, pruebas, pendientes y preguntas abiertas

Actualizado el 27 oct 2026 (con las respuestas del equipo a las 12 preguntas). Este documento resume lo que cambió en el código desde la revisión del 19 oct, cómo probarlo, qué falta y qué decisiones siguen sin respuesta. El README tiene la descripción general del proyecto.

1. Qué cambió

Decisiones ya tomadas (no se vuelven a discutir)

- **Listado final de hardware confirmado:** ESP32, Raspberry Pi 4 (4 GB) con microSD, SCD41, MAX30102, ventilador Steren 12 V con módulo D4184, módulo vibratorio 5 V, LED, buzzer **activo**, fuentes de 12 V y 5 V, protoboard, jumpers, resistencias y diodos 1N5819.
- **El AM2302 sale del proyecto.** El SCD41 ya mide temperatura y humedad. Se borró `esp32/leer_am2302.ino`.
- **El relé sigue en el circuito**, en serie entre la fuente de 12 V y el ventilador (no en paralelo con el D4184). El firmware lo usa como **interruptor de potencia**: cerrado solo cuando el ventilador debe girar (> 0 %) y abierto al arrancar y con el ventilador en 0 % (ver pregunta 1). El D4184 regula la velocidad por PWM.
- **Ventilador proporcional al CO_2. Vibración** apagada en Normal, 50 % en Alerta, 100 % en Emergencia.
- **El ESP32 ejecuta la lógica de control:** aplica los niveles que manda el servidor y, si no hay respuesta, decide solo con el CO_2 (respaldo local).

Código (commits desde el 19 oct)

Commit	Archivos	Cambio
`2762fc4`	`raspberry/logica.py`	Sin argumento mutable por defecto.
`3acc59e`	`raspberry/servidor.py`	`/lectura` responde `estado` y niveles; `/estado` incluye `estado`; nuevo `POST /operador`.
`635f307`, `72f8245`	`interfaz/`	Tarjeta de Estado, operador enviado al servidor, aviso de datos simulados, eje Y flexible, manifest sin ícono inexistente.
`267da0c`	`esp32/main_esp32.ino`	Control de actuadores con la respuesta del servidor; no envía datos inventados si falla un sensor.
`c7560db`	`raspberry/logica.py`, `servidor.py`	Ventilador proporcional al CO_2 y vibración por estado; la API devuelve `ventilador_pct` y `vibracion_pct`.
`081efca`	`interfaz/index.html`	Porcentaje del ventilador, tarjeta de vibración, `--` cuando el pulso es 0.

Commit	Archivos	Cambio
`61fae72`	`esp32/`	Firmware nuevo: SCD41 y MAX30102 por I²C, PWM del ventilador y del vibrador, LED, buzzer y respaldo local.
`0131808`, `761f706`, `fc982d6`	`docs/`, `.gitignore`, READMEs, esta guía y su PDF	Documentación actualizada y `raspberry/ia/models/` ignorado.
`35870c0`	`docs/02_Plan_Gestion_Riesgos.xlsx`	PRR 1 sin AM2302; riesgo 13 y PRR 6 reescritos (modelo de IA ausente o corrupto).
`056af0b`	`interfaz/`	Operador activo visible junto al título; Chart.js v4.4.1 local en `interfaz/vendor/` (sin CDN). Probado en Chrome contra el servidor.
`a12db28`	`raspberry/servidor.py`	Tabla `eventos` y `GET /eventos`: registra saltos de CO_2 > 150 ppm entre lecturas y cambios de estado.
`f0350f4`	`interfaz/index.html`	Lista de eventos y anomalías recientes; la gráfica carga las últimas 60 lecturas al abrir la página.
`8651e50`	`esp32/main_esp32.ino`	Relé en serie controlado por el nivel del ventilador (GPIO 32 provisional, `RELE_ACTIVO_ALTO`).
`083a3f5`	`raspberry/requirements.txt`	Versiones fijas (riesgo 15).

Reglas de salida (`raspberry/logica.py`)

Condición	Estado	Ventilador	Vibración	LED	Buzzer
CO_2 < 1000 sin riesgo	Normal	0 %	0 %	apagado	apagado
IA anticipa el umbral (< 5 min)	Alerta	30 %	50 %	encendido	apagado
CO_2 ≥ 1000 ppm	Alerta	40 % a 1000 ppm, sube lineal hasta 100 % a 2000 ppm	50 %	encendido	apagado
CO_2 > 2000, o CO_2 > 1000 con pulso > 25 % sobre 70 bpm	Emergencia	100 %	100 %	encendido	encendido

El ESP32 usa las mismas reglas con el CO_2 local (`controlLocal()`) si no hay WiFi o el servidor tarda más de 3 s.

2. Conexiones propuestas (provisionales, confirmar con Ingrid)

Elemento	Conexión	Notas
SCD41	3.3 V, GND, SDA = GPIO 21, SCL = GPIO 22	Dirección I²C 0x62.
MAX30102	3.3 V, GND, mismo SDA/SCL	Dirección 0x57. Comparte bus con el SCD41.
Ventilador (D4184)	Entrada PWM ← GPIO 26. Fuente de 12 V en el módulo; ventilador en su salida	Diodo 1N5819 en antiparalelo con el ventilador (cátodo al +12 V). Revisar la serigrafía del módulo.

Elemento	Conexión	Notas
Módulo vibratorio	Señal ← GPIO 27. Alimentación 5 V de la fuente de 5 V	Ver pregunta 3.
LED	GPIO 25 → resistencia (220 a 330 Ω) → LED → GND	
Buzzer activo	GPIO 33 → GND	Si consume más de 12 mA, usar un transistor o módulo con driver.
Relé	Contactos en serie entre la fuente de 12 V y el ventilador; control ← GPIO 32	Se propuso GPIO 14, pero emite PWM al arrancar. Cambiar `RELE_ACTIVO_ALTO` si el módulo se activa con LOW. Ver pregunta 1.
GND común	GND del ESP32, de la fuente de 12 V y de la fuente de 5 V unidos	Obligatorio antes de energizar (PRR 2).

Los pines se cambian en las constantes del inicio de `esp32/main_esp32.ino`.

3. Cómo probar

Paso 0. Preparar

```
cd raspberry
pip install -r requirements.txt
python ia/train.py        # genera ia/models/co2_model.joblib (opcional: sin él solo se usa la
regla de pendiente)
```

Paso 1. Servidor y lógica (sin hardware)

1. En una terminal: `python servidor.py` (puerto 8080). Si existe `lecturas.db` de pruebas anteriores, bórrala antes para empezar limpio.
2. Guarda este script como `prueba_api.py` (fuera del repo o en cualquier carpeta) y ejecútalo en otra terminal con `python prueba_api.py`:

```
import json, urllib.request

URL = "http://localhost:8080"

def post(ruta, datos):
    req = urllib.request.Request(URL + ruta, json.dumps(datos).encode(),
                                 {"Content-Type": "application/json"})
    return json.load(urllib.request.urlopen(req))

print(post("/operador", {"operador": "ivan"}))
for co2 in (500, 1000, 1500, 2100):
    print(co2, post("/lectura", {"co2": co2, "temp": 24, "hum": 50, "pulso": 0}))
print(json.load(urllib.request.urlopen(URL + "/estado")))
```

3. **Resultado esperado** (verificado el 24 oct, con y sin modelo entrenado):

CO_2	estado	ventilador_pct	vibracion_pct
500	Normal	0	0
1000	Alerta	40	50
1500	Alerta	70	50
2100	Emergencia	100	100

Las lecturas se acumulan en la base: la IA usa las últimas 10, así que una subida rápida (por ejemplo 500 a 900) puede dar Alerta anticipada con ventilador al 30 %. Es normal.

4. **Eventos y anomalías:** `GET /eventos` debe listar un `cambio_estado` cuando el estado cambia y un `salto_co2` cuando dos lecturas consecutivas difieren en más de 150 ppm (por ejemplo 530 a 900). Verificado el 27 oct con la secuencia 500, 520, 530, 900, 905, 1500, 1520, 800, 420: 4 saltos y 1 cambio de estado (Normal a Alerta por la IA). Con lecturas enviadas de golpe la IA ve una pendiente exagerada y puede dar Alerta anticipada; con el ESP32 real (una lectura cada 5 s) es más realista.
5. Pulso alto: envía `"co2": 1200, "pulso": 100` y debe dar **Emergencia** (pulso > 25 % sobre 70 bpm con CO_2 > 1000).
6. Entradas inválidas: `POST /lectura` con `{"co2": "abc"}` debe responder 400.

Paso 2. Dashboard

1. Con el servidor activo abre `http://localhost:8080`.
2. Tras el paso 1 debe mostrar Estado, CO_2, ventilador con su porcentaje, vibración, la gráfica con las lecturas recientes (se cargan de la base al abrir la página) y la lista de **eventos y anomalías recientes**. Sin lecturas muestra "Sin datos registrados". Probado en Chrome el 27 oct.
3. Detén el servidor: en 3 s debe aparecer **"⚠ Datos simulados (sin servidor)"**.
4. Cambia el operador en el selector y envía una lectura sin campo `operador`: `GET /ultimas` no lo muestra, pero la base guarda el operador elegido (`sqlite3 lecturas.db "select operador from lecturas order by id desc limit 1"`).

Paso 3. Firmware del ESP32 (por etapas)

Preparación en Arduino IDE:

- Instalar el soporte de placas *esp32 by Espressif* y elegir *ESP32 Dev Module*. El código acepta el core 2.x y el 3.x.
- Instalar las librerías *Sensirion I2C SCD4x* y *SparkFun MAX3010x Pulse and Proximity Sensor Library*.
- En `main_esp32.ino` poner `ssid`, `password` y la IP de la Raspberry en `serverUrl` (conviene reservar una IP fija para la Raspberry).
- **El firmware aún no se ha compilado.** Si falla al compilar, probablemente es el nombre de algún método de las librerías según su versión: avisa con el mensaje de error.

Etapas (monitor serial a 115200):

1. **Solo SCD41 + WiFi.** Debe imprimir cada 5 s `Enviando JSON: {...}` y `Respuesta Servidor: 200 {...}`. Al soplar sobre el sensor el CO_2 debe subir y aparecer en el dashboard.
2. **Agregar MAX30102.** Con el dedo sobre el sensor, `pulso` debe estabilizarse en 60 a 100 bpm en unos segundos; sin dedo debe enviar `0`. (Los latidos se detectan solo si el loop corre continuo.)

3. **Salidas sin carga.** Con un multímetro o LED en cada pin: GPIO 26 y 27 deben variar su tensión media con el CO_2 (PWM), GPIO 25 se enciende en Alerta/Emergencia, GPIO 33 solo en Emergencia y GPIO 32 (relé) debe estar activo siempre que el ventilador esté por encima de 0 %. Se puede forzar enviando lecturas desde `prueba_api.py` y mirando la respuesta en el serial.
4. **Con carga** (solo después de unir GND y poner el diodo): ventilador por el D4184 y módulo vibratorio. Verificar que el ventilador arranque con 40 % de ciclo; si no gira, subir ese mínimo en `logica.py` (y en `controlLocal()`).
5. **Respaldo local.** Apaga el servidor (o el WiFi): en el siguiente ciclo debe imprimir `Error enviando POST` o `Wi-Fi desconectado` y seguir accionando los actuadores según el CO_2.

Paso 4. Pruebas del Project Charter

Criterio	Cómo probarlo
Lecturas cada ≤ 5 s	Dejar 10 min y revisar `lecturas.db`: `select fecha from lecturas` debe mostrar saltos de ~5 s sin huecos.
Ventilador y LED en ≤ 5 s tras superar 1000 ppm	Exhalar en una caja cerrada con el sensor; cronometrar desde que el serial muestra más de 1000 ppm hasta que cambian ventilador y LED.
Anomalía provocada, registrada y visible	Exhalar sobre el sensor para provocar un salto de CO_2; debe aparecer en "Eventos y anomalías recientes" del dashboard y en `GET /eventos`. Ajustar `UMBRAL_SALTO_CO2` en `servidor.py` si con el SCD41 real no se dispara (o se dispara con el ruido normal).
Diferencia ≤ 15 % frente al monitor de referencia	Medir en paralelo con el monitor comercial y documentar la comparación.

4. Lo que falta

Software

- Compilar y depurar el firmware del ESP32 (nunca se ha compilado).
- Ajustar el umbral de anomalía (150 ppm) con datos reales del SCD41, y decidir si el pulso también cuenta como anomalía.
- Reentrenar el modelo con datos reales; hoy usa datos sintéticos.
- Si el profesor espera un detector de anomalías entrenado ("IA"), sumarlo al modelo; hoy la anomalía es una regla (salto de CO_2) y los cambios de estado incluyen las alertas que anticipa la IA.
- Crear `interfaz/icon.png` si se quiere instalar el dashboard como app.

Hardware

- Armar y revisar el circuito de potencia (PRR 2): revisarlo con un compañero antes de energizar, GND común y diodos.
- Calibrar el CO_2 del SCD41 y comparar con el monitor de referencia.

Documentos que no pude editar o requieren decisión del equipo

- `docs/01_Project_Charter_WBS.pdf` es un PDF y no lo puedo modificar. Si el Charter existe como **Google Doc**, hay que actualizarlo ahí (o pasarme el enlace y autorizarme a editarlo): lista de equipo

(agregar D4184, módulo vibratorio, fuente de 5 V, diodos, relé en serie), presupuesto de ~$1,100 MXN, líder = Iván (corregir la línea de Ingrid) y Mauricio en la portada con su matrícula.

- `docs/02_Plan_Gestion_Riesgos.xlsx`, PRR 2: la acción "*Usar módulo relé con optoacoplador*" aparece como Completada. Con el relé confirmado sigue siendo válida solo si el módulo de relé trae optoacoplador; confirmarlo con Ingrid.
- PRR 1: la acción de respaldo (STCC4 o MH-Z19B) sigue "En espera". Si el SCD41 ya llegó, se puede cerrar.
- Falta el **diagrama de conexiones** que pide el Charter (la tabla de la sección 2 es el borrador).

5. Documentación ya actualizada

- `README.md`: listado final de hardware, arquitectura, API con niveles de ventilador y vibración, reglas, instalación de librerías y comentarios para el equipo.
- `docs/02_Plan_Gestion_Riesgos.xlsx`: riesgo 3 y PRR 2 hablan de 12 V, 5 V y D4184 (tierras comunes de las dos fuentes, diodos 1N5819); PRR 1 sin el AM2302; riesgo 13 y PRR 6 reescritos como "*Modelo de IA entrenado ausente o corrupto*" (el sistema es 100 % local); nombre de Leonardo corregido a *Nájera*.
- `docs/05_Plan_Interesados.docx`: nombre de Leonardo corregido a *Nájera*.
- `docs/README.md` y `raspberry/README.md`: contenido real de las carpetas.

6. Las 12 preguntas: respuestas del equipo y qué sigue abierto

Respondidas

#	Pregunta	Respuesta	Qué se hizo
2	¿Alguien necesita el AM2302?	No, queda eliminado.	PRR 1 actualizado.
4	¿El ventilador soporta PWM?	No importa: el D4184 va entre la fuente de 12 V y el ventilador y trocea la corriente, por lo que sirve para un ventilador de 2 hilos.	Falta confirmar que el ventilador sea de 2 hilos y que arranque con 40 %.
7	¿Operador global basta?	Sí, para una demo con una persona probando a la vez.	El operador activo se ve junto al título del dashboard.
8	¿Sigue la API externa de IA?	No, todo es local (scikit-learn).	Riesgo 13 y PRR 6 reescritos como "modelo ausente o corrupto".
10	¿Internet en la demo?	Que no dependan de ello.	Chart.js ya es local. Falta fijar y anotar la IP de la Raspberry (o usar hotspot).
11	¿Quién es el líder?	Iván.	Falta corregir la línea de Ingrid en el Google Doc del Charter.
12	¿Falta Mauricio en la portada?	Sí, con su matrícula (A01779117 según su correo del Charter; confirmar).	Falta hacerlo en el Google Doc del Charter.

Respondidas con reservas (ya implementadas, con puntos por validar)

- **1. Relé.** Sigue en el circuito: en serie entre la fuente de 12 V y el ventilador, no en paralelo con el D4184. La propuesta recibida (GPIO 14 y cortar si "Emergencia" dura más de 30 s) tenía dos

problemas: cortar el ventilador en Emergencia apaga la ventilación justo cuando más hace falta, y GPIO 14 emite PWM al arrancar el ESP32 y podría activar el relé. **Lo que se implementó (8651e50, decisión mía a validar):** el relé es un interruptor de potencia, cerrado solo cuando el ventilador debe girar (> 0 %) y abierto al arrancar y con el ventilador en 0 %, en GPIO 32 (provisional). **Sigue pendiente:** el pin real, si el módulo se activa con HIGH o LOW (`RELE_ACTIVO_ALTO`), si tiene optoacoplador (PRR 2) y, si el equipo quiere además una protección contra fallas, qué la dispara y cuándo se vuelve a cerrar.

- **9. Anomalía.** Implementada con la propuesta recibida (a12db28 y f0350f4): tabla `eventos`, `GET /eventos` y lista en el dashboard; registra saltos de CO_2 > 150 ppm entre lecturas consecutivas y cambios de estado. **Sigue pendiente** decidir si el pulso cuenta, validar el umbral con el sensor real y si hace falta un detector entrenado.

Siguen abiertas (se resuelven con el circuito armado)

- **3. Módulo vibratorio:** ¿acepta PWM directo? Si es un motor suelto de 2 cables necesita transistor (y diodo); si es un módulo de 3 pines suele traer el driver (se ve un componente pequeño negro cerca de los pines).
- **5. Pines:** confirmar con multímetro antes de energizar que ventilador 26, vibrador 27, LED 25, buzzer 33 e I^2C 21/22 coinciden con lo armado.
- **6. Curva del ventilador** (40 % a 1000 ppm, 100 % a 2000 ppm, 30 % anticipado): decisión del equipo según qué tan brusco o suave se sienta en pruebas reales. Se cambia en `raspberry/logica.py` y en `controlLocal()` del firmware.
- **Frecuencia PWM:** se usan 5 kHz; un ventilador de 2 hilos puede preferir frecuencias más bajas. Probar si zumba o no gira.